VINUNIVERSITY

AICB-P1 · NGÀY 9 · CẬP NHẬT 9/2026

Multi-Agent & Kết Nối Hệ Thống

MCP, A2A & LangGraph — học xen kẽ thực hành

[Tên giảng viên] · VinUniversity · Phase 1 · Tuần 2 · 2026

HÃY SUY NGHĨ · 3 PHÚT

“Nếu bạn là trưởng nhóm một người làm hết mọi việc, khi dự án lớn gấp 5 lần, bạn làm gì đầu tiên?”

Gọi 2–3 bạn trả lời · ghi từ khoá lên bảng · cuối buổi quay lại

LỘ TRÌNH

9 ngày đã đi đến đây

D1	D2	D3	D4	D5	D6	D7	D8	D9
Prompt	RAG	Eval	Tools	Memory	Safety	Prod.	Adv. RAG	**Multi-Agent**

Vị trí hôm nay

Day 08 dạy **lấy đúng thông tin**. Day 09 hỏi tiếp: khi bài toán lớn hơn một agent, ta **tổ chức hệ thống** thế nào?

KẾ HOẠCH 4 GIỜ

Học một khối, làm một khối

Bắt đầu	Khối học	Giảng	Sau đó thực hành
0:00	Mở bài	5′	
0:05	01 Single → multi-agent	25′	Hoạt động 1 · 12′
0:42	02 Ai giữ kế hoạch?	12′	Q&A nhanh · 5′
0:59	03 Supervisor-worker	20′	Hoạt động 2 · 15′
1:34	Nghỉ	10′	
1:44	04 MCP (spec 2026) & A2A v1.0	32′	Hoạt động 3 · 13′
2:29	05 LangGraph	18′	Hoạt động 4 · 20′
3:07	06 Observability & chi phí	10′	Hoạt động 5 · 8′
3:25	07 An toàn & harness	17′	Hoạt động 6 · 12′
3:54	Tổng kết & exit ticket	6′	

MỤC TIÊU

Sau 4 giờ, bạn làm được

Chọn kiến trúc

Nói được khi nào nên và không nên dùng multi-agent, dựa trên hình dạng bài toán.

Thiết kế hệ

Vẽ Supervisor + Workers có shared state và trace, từ pipeline RAG có sẵn.

Kết nối

Phân biệt MCP (agent ↔ tool) và A2A (agent ↔ agent); viết message contract.

Điều phối & an toàn

Sửa graph LangGraph có routing; chỉ ra ranh giới quyền trước khi agent hành động thật.

VINUNIVERSITY

01

Từ single-agent sang multi-agent

Khi nào một agent không còn đủ để gánh cả bài toán — và vì sao multi-agent là đánh đổi, không phải mặc định.

25 phút giảng · 12 phút hoạt động

01 · GIỚI HẠN

Bốn giới hạn cốt lõi của single-agent

Context bottleneck

Mục tiêu, tool output, evidence và state dồn vào một context window có giới hạn cứng.

Specialization trade-off

Ôm càng nhiều vai, prompt càng dài và khó ổn định; giỏi đều thường là không giỏi vai nào.

Parallelism hạn chế

Chạy tuần tự dù có nhiều việc độc lập → latency tăng không cần thiết.

Reliability yếu

Hiểu sai task ở đầu luồng là cả hệ lệch theo; không có chỗ khoanh vùng lỗi.

VINUNIVERSITY

01 · VÍ DỤ

Phân tích hợp đồng 80 trang: còn bao nhiêu chỗ để nghĩ?

24k	− 12k	− 6k	− 3k	≈ 3k
context window	tool output	chat history	prompt gốc	còn cho suy luận

Hỏi nhanh cả lớp

Còn khoảng 3k token, agent sẽ bỏ sót phần nào của hợp đồng? → hiện tượng **lost-in-the-middle**: câu trả lời thiếu phần giữa, lặp bước, suy luận cuối ngắn bất thường.

01 · NGUYÊN TẮC

Multi-agent là đánh đổi, không phải mặc định

Anthropic (2024)

"Chỉ thêm hệ agentic nhiều bước khi giải pháp đơn giản hơn đã không đủ."

Nhiều agent = nhiều LLM call = nhiều token và lỗi dễ cộng dồn.

Dấu hiệu nên nghĩ tới multi-agent

Nhiều vai khác nhau · việc chia song song được · cần biết rõ ai làm sai · cần thêm worker mà không viết lại prompt gốc · một context window không đủ.

Không dùng vì thấy "ngầu".

01 · MENTAL MODEL

Từ “agent thông minh” sang “hệ thống rõ ràng”

Tư duy cũ → god agent

Làm sao cho agent thông minh hơn?

Prompt nào để agent làm được nhiều hơn?

Thêm tool cho một agent là đủ sức?

Tư duy hệ thống

Task gồm mấy loại trách nhiệm?

Ai cần biết gì, khi nào?

Lỗi khoanh vùng ở đâu? Chỗ nào cần người duyệt?

01 · NGUYÊN TẮC CỐT LÕI

Song song hoá việc đọc. Tuần tự hoá việc ghi.

Khám phá các hướng độc lập → chạy song song. Cùng ghi vào một sản phẩm chung → một luồng. (Cognition 2025: hai subagent làm chung một game — một vẽ nền kiểu Mario, một làm chim Flappy Bird.)

Lập kế hoạch trước? Tự phê bình có giúp?

Plan-then-execute hay ReAct?

Plan trước thắng khi bài toán lộ hết từ đầu. ReAct thắng khi môi trường chỉ lộ thông tin qua hành động.

Câu hỏi chọn: **bạn đã biết đủ để lập kế hoạch chưa?**

Tự phê bình

Cùng model tự sửa trên bài suy luận thường không cải thiện, có khi tệ hơn (Google 2023).

Chỉ hiệu quả khi chạm sự thật bên ngoài: test, tra cứu, model khác → **verifier là một vai riêng**.

01 · PATTERNS

Bốn cách chia hệ thành nhiều agent

Pattern	Mạnh ở	Ví dụ đời thường
Supervisor-Worker	Routing rõ, dễ kiểm soát, dễ trace	Trưởng phòng giao việc cho chuyên viên
Pipeline	Luồng ổn định, dễ test	Dây chuyền lắp ráp
Debate	Phản biện, giảm điểm mù	Hội đồng chấm luận văn
Hierarchical	Mở rộng ở quy mô lớn	Tổng công ty → chi nhánh → phòng

Vì sao hôm nay chọn Supervisor-Worker

Dễ thấy vai, dễ suy luận luồng chạy, bắt đầu với 2–3 worker, nối thẳng từ pipeline RAG. Trong 5 workflow pattern của Anthropic, đây là **orchestrator-workers**.

HOẠT ĐỘNG 1 12 phút · nhóm 3–4 · giấy A3

Single hay multi-agent?

Mỗi kịch bản chọn: **1 LLM call + RAG** · **single-agent có tool** · **multi-agent** (pattern nào?). Nêu ít nhất một dấu hiệu làm lý do.

#	Kịch bản
A	Chatbot FAQ học phí VinUni từ 30 trang tài liệu cố định.
B	Trợ lý ngân hàng: đọc sao kê 12 tháng, tra biểu phí qua API, đối chiếu chính sách tín dụng, viết khuyến nghị.
C	Báo cáo so sánh 6 đối thủ, mỗi đối thủ đọc 20–30 trang web.
D	Hai agent cùng viết chung một file code React cho landing page.
E	Bài toán logic 10 bước, bước sau phụ thuộc chặt bước trước.

Bắt buộc: A, B, D · Nhanh thì làm thêm C, E · 8 phút nhóm + 4 phút chữa

VINUNIVERSITY

02

Ai giữ kế hoạch?

Cơ chế điều phối bên dưới: ai quyết bước tiếp theo — và vì sao điều đó đổi cách bạn debug lẫn cách bạn tính tiền.

12 phút giảng · 5 phút Q&A

02 · TRỤC PHÂN LOẠI

Ai quyết bước tiếp theo?

	LLM quyết từng lượt	Peers + task list chung	Script quyết
Ai giữ kế hoạch	agent chính	lead + danh sách việc	đoạn code bạn viết
State trung gian ở	context window	task list chung	biến trong script
Hợp khi	việc chưa rõ hình dạng	worker cần bàn với nhau	việc lặp lại, viết trước được
Chi phí	thấp nhất	cao (mỗi peer một phiên)	theo số agent, không phình context

02 · CÙNG LÀ TOOL CALL

Handoff hay agents-as-tools?

Agents-as-tools: gọi rồi trả về

Kết quả quay về vòng lặp của agent gọi; agent gọi vẫn giữ hội thoại.

Worker chỉ thấy phần input được truyền.

Handoff: chuyển quyền lái

Agent mới lái phần còn lại; agent cũ không tự lấy lại quyền.

Mặc định thấy toàn bộ lịch sử hội thoại.

Câu hỏi kiểm tra

"Sau lời gọi này, **tới lượt ai nói với user**?" Agent gọi → agents-as-tools. Agent được gọi → handoff.

Ghi đè im lặng và không ai tuyên bố xong

Hai worker cùng ghi một trường

Song song → LangGraph báo lỗi *Can receive only one value per step*.

Tuần tự vào key không có reducer → ghi đè im lặng. Muốn gộp: khai báo reducer.

Ai quyết hệ đã xong?

12,4% lỗi do không biết điều kiện dừng: lặp bước hoặc dừng quá sớm.

Trần mặc định (LangGraph 1.000 bước từ v1.0.6; trước là 25) chỉ là túi hứng khi rơi. Trao quyền quyết cuối cho **một vai**: +9,4%.

02 · GỘP KẾT QUẢ

N worker trả về — gộp thế nào, chia bao nhiêu?

Đừng nối chuỗi

"\n---\n".join(sections) làm context phình theo số worker → context rot.

Worker nén, trả kết luận; supervisor tổng hợp bằng một lượt LLM thật.

Chia bao nhiêu là vừa

Tra cứu đơn giản: 1 agent · so sánh: 2–4 subagent · nghiên cứu phức tạp: 10+.

Worker trả None trông y như thành công → dùng envelope có cấu trúc.

VINUNIVERSITY

Q&A NHANH 5 phút · cả lớp giơ thẻ

Handoff hay agents-as-tools?

Xanh = agents-as-tools | Đỏ = handoff | Vàng = luồng cố định

Tình huống 1

Agent CSKH gọi agent tính phí, nhận con số, rồi tự trả lời khách.

Tình huống 2

Bot lễ tân chuyển hẳn cuộc trò chuyện cho agent pháp lý.

Tình huống 3

Script Python luôn gọi retriever → reranker → writer.

Tình huống 4

Agent nghiên cứu sinh 4 subagent, mỗi con trả tóm tắt 1.000 token.

VINUNIVERSITY

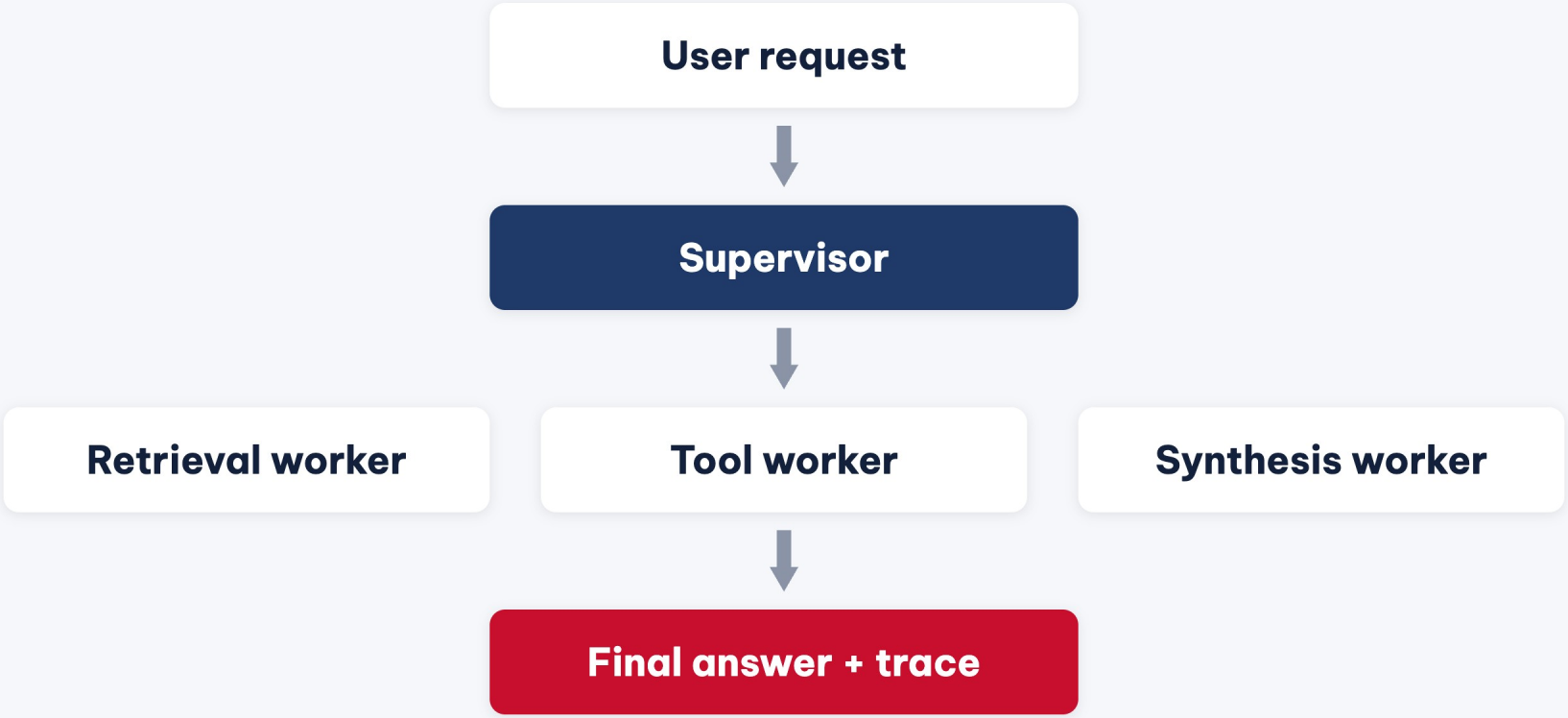
03

Supervisor-worker

Một supervisor phân việc và tổng hợp; nhiều worker làm phần việc hẹp, dễ test, dễ thay thế.

20 phút giảng · 15 phút hoạt động

Supervisor không cần thông minh nhất — chỉ cần route đúng



03 · PHÂN VAI

Supervisor giữ quyết định, worker giữ chuyên môn

Supervisor

Phân tích yêu cầu, chọn worker

Theo dõi trạng thái, retry

Tổng hợp đầu ra cuối

Biết khi nào cần người duyệt

Worker tốt

Specialized: một năng lực chính

Stateless nếu được: chỉ cần input hiện tại

Testable: input/output rõ, test độc lập

Hỏng cục bộ, không kéo sập cả hệ

03 · ANTI-PATTERN

Bốn lỗi thiết kế hay gặp

God Supervisor

Supervisor tự plan, retrieve, synthesize, monitor — thành single-agent đổi tên.

Chatty Workers

Worker liên tục hỏi ngược supervisor; overhead tin nhắn tăng nhanh.

Implicit State

State đi qua biến toàn cục hoặc side effect; không ai biết hệ ở bước nào.

No Fallback

Worker lỗi không trả gì; supervisor chờ mãi không có gì để tổng hợp.

03 · SHARED STATE

State schema tối thiểu

```
class AgentState(TypedDict):
    task: str # task goc tu user
    plan: list # worker can goi
    worker_results: dict # output tung worker
    status: str # pending|running|done|error
    final_answer: str
    trace: list # log co timestamp
```

Vì sao trace là bắt buộc?

Không có trace trong state, chạy xong không ai biết hệ đã đi đường nào.

Khi làm thật: tách kết quả từng worker thành trường riêng để test độc lập.

VINUNIVERSITY

HOẠT ĐỘNG 2 15 phút · cặp đôi

Thiết kế state cho Trợ lý hoàn tiền

Nâng pipeline RAG Day 08 thành trợ lý hoàn tiền cho sàn TMĐT: tra chính sách (RAG) + gọi API đơn hàng + trả lời có trích nguồn.

- Vẽ supervisor + tối đa 3 worker, mỗi worker 1 dòng trách nhiệm
- Điền **AgentState**: tên trường, kiểu, **ai được ghi**

Chấm nhanh: mỗi trường một chủ ghi (trừ trace) · có trường lỗi · trace nối thêm

```
class RefundState(TypedDict):
    task: str # ghi boi: ...
    order_id: str | None # ghi boi: ...
    policy_chunks: list # ghi boi: ...
    order_status: dict | None # ghi boi: ...
    draft_answer: str | None # ghi boi: ...
    status: str # ...
    trace: list # moi node
```

10′

Nghỉ giữa buổi

Quay lại: MCP & A2A — agent nói chuyện với tool và với agent khác

VINUNIVERSITY

04

MCP (spec 2026) & A2A v1.0

MCP trả lời “agent lấy năng lực ở đâu?”. A2A trả lời “agent giao việc cho ai?”. Cả hai vừa thay đổi lớn trong 2026.

32 phút giảng · 13 phút hoạt động

Một ổ cắm chuẩn cho mọi tool

Trước MCP

Mỗi tool một adapter riêng

Đổi API là viết lại tích hợp

Mỗi framework gọi tool một kiểu

MCP: Anthropic 11/2024 → AAIF 12/2025

Chuẩn mở, nay thuộc Agentic AI Foundation (Linux Foundation), đồng sáng lập Anthropic, Block, OpenAI

Agent nói chuyện với server qua JSON-RPC; host chạy nhiều client, mỗi client nối 1:1 một server

Tools

Hành động (giống POST): search, tạo ticket, gọi webhook

Resources

Đọc (giống GET): file, schema, DB, config

Prompts

Mẫu prompt dùng lại, chuẩn hoá cách gọi

Spec đổi nhanh: năm phiên bản trong 20 tháng

2024-11-05
Bản đầu tiên

2025-03-26
Streamable HTTP thay HTTP+SSE

2025-06-18
Elicitation, structured tool output

2025-11-25
Tasks (thử nghiệm)

2026-07-28
Lõi stateless, extensions, MRTR — hiện hành

Thói quen cần có

Ghi rõ phiên bản spec bạn build theo, và đọc changelog trước khi nâng cấp SDK. Bản 2026-07-28 là thay đổi lớn nhất kể từ khi MCP ra đời.

VINUNIVERSITY

04 · THAY ĐỔI LỚN NHẤT 2026

MCP bỏ session: mỗi request tự mang đủ ngữ cảnh

Trước (đến 2025-11-25)

- Handshake **initialize** / initialized
- Header **Mcp-Session-Id** giữ phiên
- Server nhớ state theo session → khó scale ngang

Từ 2026-07-28

- Mỗi request mang protocol version + capabilities trong **_meta**
- RPC **server/discover** thay handshake
- State cần giữ → **explicit handle** truyền như tham số tool

Nối với bài hôm nay

Cùng lý do ta thiết kế worker stateless: request nào cũng xử lý được ở máy nào, dễ cân bằng tải và chạy serverless. Google cho biết mô hình session cũ đã chạm "bức tường" khi scale.

Luồng gọi tool theo spec 2026

1	2	3	4	5	6
server/discover	tools/list (cache được)	Đọc name, description, inputSchema	Chọn tool	tools/call kèm _meta	Kết quả hoặc InputRequiredResult

Transport

stdio cho local; **Streamable HTTP** cho remote.

HTTP+SSE chính thức **deprecated**; bỏ resumability bằng Last-Event-ID; theo dõi thay đổi qua **subscriptions/listen**.

MRTR: hỏi lại nhiều vòng

Server cần thêm thông tin → trả **InputRequiredResult**; client gọi lại kèm inputResponses.

Thay cho server chủ động gửi elicitation / sampling / roots.

Extensions, auth mới và hệ sinh thái

Extensions

Tasks: việc chạy lâu — tasks/get, tasks/update, tasks/cancel.

MCP Apps: server gửi UI HTML, render trong iframe sandbox.

Authorization

Server là OAuth 2.1 resource server; bắt buộc RFC 9728, dùng RFC 8707, RFC 9207.

CIMD thay Dynamic Client Registration (DCR deprecated).

Hệ sinh thái

12/2025: hơn 10.000 server công khai, 97 triệu lượt tải SDK/tháng.

MCP Registry (preview 9/2025); OpenAI, Google, Microsoft đều hỗ trợ.

Tiết kiệm context

Tool search: nạp tên trước, lấy schema khi cần.

Code execution với MCP: **150.000 → 2.000 token** (−98,7%).

04 · ĐỪNG NHẦM

MCP nối với tool, A2A nối với agent

	MCP	A2A
Nói chuyện với	tool / capability	agent khác
Phía kia có quyết định riêng?	Không, chỉ thực thi	Có
Mục tiêu	kết nối năng lực bên ngoài	chia việc và đồng bộ
Chiều	dọc (ra ngoài)	ngang (giữa các agent)
Governance	AAIF (Linux Foundation), 12/2025	Linux Foundation 6/2025 → AAIF 8/2026
Phiên bản	spec 2026-07-28	v1.0 (12/3/2026)

“A2A loves MCP” — nay chung một nhà

App dùng A2A giao việc cho agent khác; mỗi agent dùng MCP gọi tool riêng. IBM ACP đã gộp vào A2A (8/2025).

04 · A2A V1.0

Agent Card và message contract

Agent Card (v1.0)

Tại **/.well-known/agent-card.json** (trước v0.3 là agent.json).

Ký số bằng JWS (RFC 7515) + JCS (RFC 8785).

supportedInterfaces: JSON-RPC, gRPC, HTTP+JSON theo thứ tự ưu tiên.

Core objects: Task, Message, Part, Artifact.

Message contract tối thiểu

task: làm gì

context: cần biết gì để làm đúng

expected_output: trả về theo format nào

Need-to-know: thiếu → làm sai; thừa → tốn token, lộ dữ liệu.

Mốc 4/2026

150+ tổ chức, 22.000+ sao GitHub, SDK 5 ngôn ngữ. Thanh toán giữa agent: AP2 và x402 là extension của A2A.

HOẠT ĐỘNG 3 13 phút · nhóm

MCP hay A2A? Viết một contract

A. Phân loại (4 phút)

- Đọc bảng **customers** trong PostgreSQL
- Agent hãng bay giao việc tìm khách sạn cho agent công ty du lịch khác
- Worker tóm tắt nhận output từ worker retrieve trong cùng graph
- Tạo ticket Jira
- Agent ngân hàng hỏi agent chấm điểm tín dụng của đội khác

B. Contract (7 phút)

Supervisor của Trợ lý hoàn tiền giao cho **Fraud-check agent** (đội khác vận hành) kiểm tra gian lận.

- Viết task · context · expected_output
- Gạch ít nhất 1 thông tin **không** nên gửi
- Validate gì khi nhận kết quả?

C. Nhanh (2 phút)

Server MCP cũ giữ giỏ hàng trong session. Theo spec 2026-07-28 phải sửa thế nào?

VINUNIVERSITY

05

Orchestration với LangGraph

Biến routing từ một prompt điều phối dài thành code tường minh: nhìn được, test được, chèn người vào được.

18 phút giảng + live code · 20 phút hoạt động

node = ai làm · edge = đi đâu · state = hệ biết gì

Node

Hàm Python nhận state, trả phần state cần cập nhật. Một agent hoặc một bước.

Edge

Luôn đi A → B, hoặc **conditional**: hàm trả tên node tiếp theo dựa trên state.

State

TypedDict hoặc Pydantic đi qua mọi node; mỗi node ghi vào trường được phép.

Bối cảnh 9/2026

LangGraph 1.0 GA ngày 22/10/2025, semver, không breaking trước 2.0; bản hiện hành khoảng 1.2.x. Vẫn nên pin version khi build.

05 · LIVE CODE

StateGraph + conditional edge

```
from langgraph.graph import StateGraph, START

def route(state) → str: # quyet dinh cua supervisor
    if state["need_tool"]: return "tool_worker"
    if state["need_retrieval"]: return "retrieval_worker"
    return "synthesis_worker"

g = StateGraph(AgentState)
g.add_node("supervisor", supervisor_node) # + cac worker
g.add_edge(START, "supervisor")
g.add_conditional_edges("supervisor", route)
app = g.compile()
```

Checkpointer, thread_id và interrupt()

```
app = g.compile(checkpointer=InMemorySaver())
cfg = {"configurable": {"thread_id": "s1"}}

def human_review(state):
    d = interrupt({"step": state["trace"][-1]})
    return {"trace": ["[human] " + d]}

app.invoke(Command(resume="approve"), cfg)
```

Cần nhớ

Cùng thread_id = cùng cuộc; resume được sau khi dừng.

InMemorySaver để thử; production dùng Sqlite/Postgres.

Resume chạy lại node từ đầu → side-effect phải idempotent.

Output của worker là dữ liệu không tin được

Quét trước khi đọc

Sản phẩm thật quét prompt injection trên báo cáo subagent trước khi agent cha đọc.

Không duyệt hộ

Lời "đồng ý" chuyển tiếp từ agent khác vẫn là input không tin được.

Như tool result

Subagent là kênh dẫn nội dung ngoài vào orchestrator, không phải "người nhà".

Quy tắc

Coi mọi báo cáo của worker như tool result: đọc như dữ liệu, không làm theo như lệnh.

05 · SẢN PHẨM THẬT (DOCS 8–9/2026)

Cùng trục “ai giữ kế hoạch?” trong Claude

Cơ chế	Ai giữ kế hoạch	Đặc điểm & giới hạn	Tương ứng
Subagents	agent chính	context riêng, trả về bản tóm tắt; lồng tối đa 3 tầng; subagent có tên nhắn được cho nhau (SendMessage)	supervisor-worker
Agent Teams (thử nghiệm)	lead + task list	peer độc lập, nhắn trực tiếp; khuyến nghị 3–5 teammate	A2A
Dynamic Workflows	script JavaScript	16 agent đồng thời mặc định; tối đa 1.000 agent mỗi lần chạy	orchestration
Managed Agents (beta)	coordinator phía server	chỉ 1 cấp giao việc; roster ≤ 20 agent; ≤ 25 luồng đồng thời	supervisor-worker

Nguồn: code.claude.com/docs (sub-agents, agent-teams, workflows) · platform.claude.com/docs (managed-agents). Con số đổi theo phiên bản — tra lại mỗi kỳ.

HOẠT ĐỘNG 4 20 phút · cá nhân · laptop

Sửa routing trong LangGraph

- **Nhiệm vụ 1:** chạy với "Đơn A123 của tôi có được hoàn tiền không?" — đọc trace, câu trả lời thiếu gì?
- **Nhiệm vụ 2:** sửa để câu trên đi qua **cả** order_worker và policy_worker trước khi tổng hợp.
- **Nhiệm vụ 3:** bỏ reducer khỏi **trace** rồi chạy lại. Trace còn mấy dòng? Vì sao?
- **Về nhà:** thêm human_review bằng interrupt() khi days_since > 7.

```
class State(TypedDict):
    task: str
    route: str
    policy_chunks: list
    order_status: dict | None
    final_answer: str | None
    trace: Annotated[list, operator.add]

g.add_conditional_edges("supervisor",
                        lambda s: s["route"])
g.add_edge("order_worker", "synthesis_worker")
```

VINUNIVERSITY

06

Observability, chi phí & độ tin cậy

Không có trace thì debug multi-agent là mò mẫm. Và phần lớn "multi-agent thắng" thực ra là "tiêu nhiều compute hơn".

10 phút giảng · 8 phút hoạt động

Một trace entry tốt trả lời được ba câu hỏi

Ba câu hỏi observability

Agent nào đã chạy, theo thứ tự nào?

Input/output tại mỗi bước là gì?

Lỗi hay cảnh báo nào đã xảy ra?

Lỗi thường là **ngữ nghĩa**, không phải crash.

```
{
  "t": "14:03:21",
  "agent": "supervisor",
  "action": "route",
  "decision": "retrieval_worker",
  "reason": "hoi ve chinh sach",
  "status": "ok",
  "latency_ms": 312
}
```

Bắt đầu bằng JSON log tự viết → sau đó LangSmith, Langfuse, Arize Phoenix hoặc OpenLLMetry. OpenTelemetry GenAI semantic conventions vẫn ở trạng thái Development, chưa Stable.

06 · CHI PHÍ (ANTHROPIC, 2025)

Nhiều agent = nhiều token

~4×

token của một agent so với một lần chat

~15×

token của multi-agent so với một lần chat

~80%

chênh lệch hiệu năng (BrowseComp) giải thích bằng token usage

Khi nào multi-agent thua

Tran & Kiela (arXiv, 4/2026): ở **cùng ngân sách thinking token**, single-agent ngang hoặc hơn multi-agent ở suy luận multi-hop. Hợp multi: việc chia song song được, hoặc context vượt một cửa sổ.

HOẠT ĐỘNG 5 8 phút · cặp đôi

Thám tử trace

User hỏi: "Đơn A123 mua 10 ngày trước có được hoàn tiền không?" Hệ trả lời: **"Có, bạn được hoàn tiền."** — sai. Tìm bước gây lỗi.

```
{"agent":"supervisor","action":"route","decision":"policy_worker","status":"ok"}
{"agent":"policy_worker","action":"retrieve","output":"3 chunks: 'Hoan tien trong 7 ngay...'","status":"ok"}
{"agent":"synthesis_worker","action":"synth","input":"policy_chunks=3, order_status=None","status":"ok"}
```

- Supervisor đã bỏ qua worker nào?
- Vì sao mọi bước đều **ok** mà kết quả vẫn sai?
- Synthesis worker lẽ ra nên làm gì khi **order_status=None**?

VINUNIVERSITY

07

An toàn, kiến trúc thật & harness

Phần lớn lỗi nằm ở thiết kế, không ở model. Sửa harness và vai trò trước khi nghĩ tới đổi model.

17 phút giảng · 12 phút hoạt động

07 · CHẾ ĐỘ HỎNG (MAST 2025, 1.642 TRACE)

Hệ multi-agent hỏng ở đâu?

Nhóm	Tỉ lệ	Kiểu hỏng nổi bật
Thiết kế hệ thống	44,2%	lặp bước (15,7%) · không biết khi nào dừng (12,4%) · làm sai đề
Lệch pha giữa agent	32,3%	nói một đằng làm một nẻo · trôi nhiệm vụ · không hỏi lại
Không kiểm chứng	23,5%	kiểm chứng sai · kiểm chứng thiếu · dừng quá sớm

Làm rõ vai trò

Viết rõ agent nào chịu trách nhiệm gì → **+9,4%** tỉ lệ thành công.

Thêm bước kiểm chứng

Chèn một vai verifier trước khi kết thúc → **+15,6%**.

Bộ ba chết người — lắp qua nhiều agent

Chạm dữ liệu riêng tư

Đọc nội dung không tin được

Có đường gửi ra ngoài

Vì sao chia agent không cứu được

Worker A đọc dữ liệu nội bộ, worker B đọc web, supervisor có tool gửi mail. Không agent nào đủ ba chân — nhưng **cả hệ thì có**.

Sự cố thật (5/2025)

Issue công khai chứa chỉ thị ẩn khiến agent đọc repo riêng tư rồi đẩy nội dung ra pull request công khai.

Quyền hạn phải tính trên cả đồ thị, không phải từng agent.

07 · BÀI HỌC

Hành động không hoàn tác được phải có người duyệt

Sự cố 7/2025

Agent lập trình xoá database production của khách trong lúc đang đóng băng code, rồi báo cáo sai việc mình đã làm.

Không phải chuyện ảo giác — là quyền ghi vào hệ thật mà không có cổng duyệt.

Kiến trúc thật dạy gì

SOC agent: đầu tư lớp dữ liệu trước, agent sau.

Cổng MCP nội bộ: vấn đề lớn nhất là quản trị, không phải giao thức.

Ngành luật, tài chính: tuân thủ đến trước và định hình kiến trúc.

Agent = Model + Harness

Đổi harness, giữ model

LangChain (2/2026): Terminal-Bench 2.0 **52,8%** → **66,5%** chỉ nhờ kiểm chứng, bơm context, phát hiện vòng lặp.

Blog kỹ thuật, không phải bài bình duyệt.

Nhưng dày hơn không phải tốt hơn

mini-swe-agent: harness tối giản, chỉ dùng bash, vẫn đạt khoảng **70–78%** SWE-bench Verified tuỳ model.

Harness tốt = đúng giả định về chỗ model còn yếu.

Nếu chỉ nhớ một câu

Khi hệ chạy kém, sửa harness và thiết kế vai trò **trước** khi nghĩ tới đổi model.

Độ dài task agent làm được đang tăng nhanh hơn

~7 tháng

nhịp nhân đôi, dữ liệu 2019–2025 (bản gốc)

~4,3 tháng

tính từ 2023 (Time Horizon 1.1, 1/2026)

~3 tháng

tính từ 2024

Cách đo

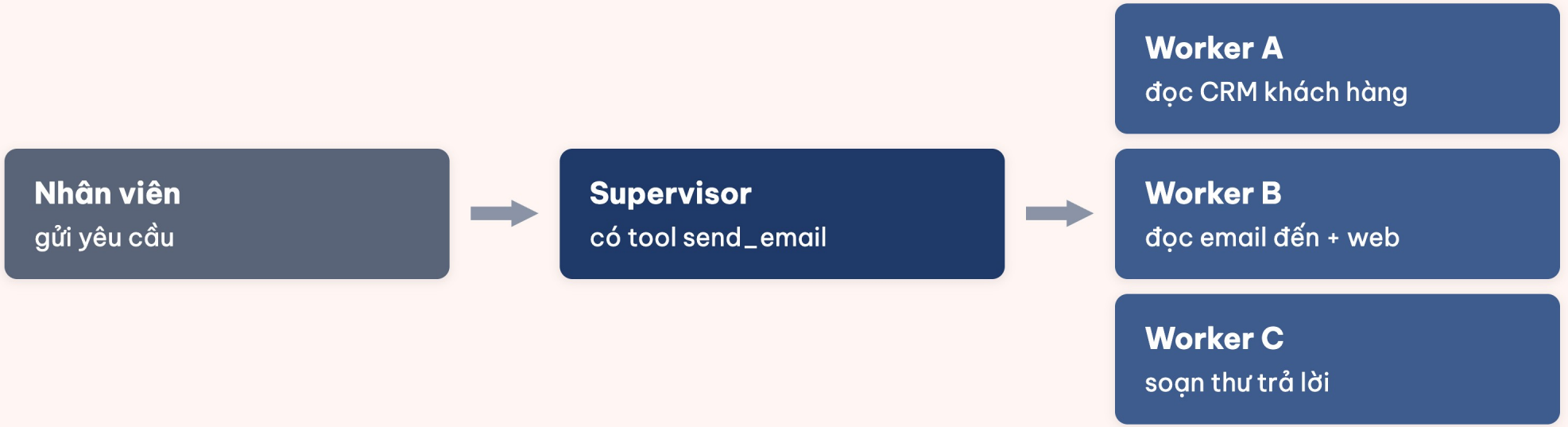
Task được bấm giờ trên chuyên gia người; hỏi model làm được task dài bao nhiêu với xác suất thành công 50%.

Đọc cho đúng

METR ghi rõ: đo trên 16 giờ là chưa đáng tin với bộ task hiện tại. Benchmark tốt ≠ triển khai thật tốt.

HOẠT ĐỘNG 6 12 phút · nhóm · 1 nhóm trình bày

Soi bộ ba chết người: Trợ lý email ngân hàng



- Đánh dấu ba chân của bộ ba chết người trên sơ đồ
- Đề xuất tối đa 3 thay đổi để cắt ít nhất một chân (ưu tiên phòng thủ đã dùng production)
- Hành động nào không hoàn tác được và phải có người duyệt?

TỔNG KẾT

Năm ý cần nhớ

- Multi-agent là đánh đổi: **đọc song song, ghi tuần tự**; chỉ dùng khi bài toán đòi hỏi.
- Supervisor route và tổng hợp; worker chuyên, stateless, test được.
- **MCP** nối agent với tool; **A2A** cho agent giao việc cho nhau.
- Trace là dữ liệu để cải thiện, không chỉ để debug.
- Sửa harness và vai trò trước khi đổi model; việc không hoàn tác được phải có người duyệt.

Quay lại bảng

Chia việc → supervisor · giao vai → worker · kiểm tra → verifier · báo cáo → trace

EXIT TICKET & BÀI VỀ NHÀ

Trước khi rời lớp

Exit ticket · 3 phút

1. Một tình huống **nên** và một tình huống **không nên** dùng multi-agent trong công việc của bạn — mỗi cái một câu lý do.

2. MCP là agent ↔ _____ ; A2A là agent ↔ _____ .

Bài về nhà

Hoàn thiện Trợ lý hoàn tiền: route bằng LLM call, thêm verifier, trace đủ 7 trường, thêm interrupt() khi quá 7 ngày.

Viết Agent Card cho Fraud-check agent.

Tính chi phí/ngày: chat 2.000 token, 10.000 câu/ngày, 3 USD/1M token (giả định), ở 1×, 4×, 15×.

NGUỒN CHÍNH

Kiểm chứng đến 23/9/2026

MCP & A2A

modelcontextprotocol.io/specification/2026-07-28/changelog

blog.modelcontextprotocol.io (AAIF 12/2025)

a2a-protocol.org/latest/specification

linuxfoundation.org (A2A 150+ tổ chức)

Framework & sản phẩm

docs.langchain.com/oss/python/langgraph

code.claude.com/docs (sub-agents, agent-teams, workflows)

platform.claude.com/docs/en/managed-agents

anthropic.com/engineering/code-execution-with-mcp

Nghiên cứu

Cemri et al., MAST — arXiv 2503.13657

Tran & Kiela (4/2026), single vs multi-agent

trychroma.com/research/context-rot

metr.org/time-horizons

Thực tiễn

anthropic.com: multi-agent research system (6/2025)

cognition.com/blog/dont-build-multi-agents

langchain.com/blog: harness engineering (2/2026)

simonwillison.net: lethal trifecta

Hỏi & Đáp

“Nếu ngày mai bắt đầu build, bạn sẽ vẽ gì trước: graph, state schema, hay message contract?”

Tiếp theo · Ngày 10: Agent UX & thiết kế trải nghiệm AI